HEP Agent Simulation Framework

Model Documentation

June 1, 2026

Contents

Chapter 1

Introduction

This document describes the mathematical and logical model underlying the Human Ecology Project (HEP) Agent Simulation.

Chapter 2

Environment Model

2.1 Grid Structure

Description of the spatial grid, resolution, and coordinate system.

2.2 Habitat Suitability (HEP)

Description of the HEP index and how it influences agent survival and movement.

Chapter 3

Agent Dynamics

3.1 Population Structure

Agents are defined by a set of intrinsic and extrinsic attributes:

- **Age**: Tracked in ticks (1 tick $\approx$ 1 week).
- **Gender**: Binary classification (Male/Female).
- **Resources**: Energy levels accumulated from grid cells.

3.2 Movement Rules

Logic governing agent migration and dispersal.

3.3 Demographics

The demographics model manages population growth and mortality through a combination of individual agent rules and macroscopic feedback loops.

3.3.1 Birth Model (Verhulst Pressure)

To ensure population stability and prevent exponential runaway, the simulation utilizes a **macroscopic fertility scaling factor** (`K_fertility`).

The target birth rate ϕ_{target} is calculated using the Verhulst logistic equation:

$$\phi_{target} = r \cdot \left(1 - \frac{N}{N_C}\right) \tag{3.1}$$

Where:

- r: Intrinsic growth rate.
- N: Current total population count.
- N_C: Carrying capacity of the environment.

The actual fertility of each female agent is then scaled by $K_{fertility}$ to match this target, bounded by `Kmin` and `Kmax` to prevent demographic collapse or excessive volatility.

3.3.2 Death Model

Mortality is driven by multiple factors:

- **Natural Mortality**: Age-dependent risk.
- **Starvation**: Triggered when an agent's resources fall below a critical threshold.
- **Environmental Risk**: Probability of death based on HEP suitability (e.g., drowning in water cells).

Chapter 4

Interaction Model

4.1 Resource Competition

How agents compete for limited resources in a grid cell.

Chapter 5

Recent Changes

Chapter 6

Project State Summary - May 5, 2026

6.1 Objective

The primary goals addressed recently were the implementation of **Auto K-Means and DBSCAN**, resolving simulation stability issues, performing a general codebase cleanup, and standardizing the **Birth-Death simulation modules** for consistent scientific modeling.

6.2 Completed Work

6.2.1 1. Dead Agent Archive & NetCDF Storage (April 21st)

- **Objective**: Implement a performance-conscious system for archiving deceased agents for historical analysis.
- **Work Done**:
  - **Fortran Backend**: Added `store_dead_agents` flag to `world_container` and logic to archive agent data before deletion.
  - **Asynchronous Pipeline**: Built an async Python writer using `multiprocessing.Queue` to persist data to NetCDF without blocking the simulation thread.
  - **UI Integration**: Added a checkbox in the "Full Simulation" tab and status indicators for the writer queue.
  - **Bug Fixes**: Resolved segfaults in the F2PY interface by refactoring agent extraction into a clean dataclass-based pipeline.

6.2.2 2. New Module Pattern & World Data Access (April 23rd)

- **Objective**: Standardize how new agent-logic modules are added and how they interact with global state.
- **Work Done**:
  - **Accumulator Reset**: Established a mechanism for `t_tick_accumulators` that automatically reset at the end of each simulation tick.
  - **Module Registration**: Defined a clear 3-step pattern for adding modules (Fortran ID -> Python Interface -> SpawnPointEditor registration) following the `reviewed_agent_motion` template.
  - **World Access**: Identified and documented accessible agent variables via the `world` object to facilitate custom agent-logic development.

6.2.3 3. DBSCAN, Convex Hull & Performance Metrics (April 28th)

- **Objective**: Expand clustering options, improve cluster integrity, and track simulation efficiency.
- **Work Done**:
  - **DBSCAN Implementation**: Added point-based and grid-based DBSCAN clustering in Fortran.
  - **UI Controls**: Added dynamic control over DBSCAN parameters (`eps`, `minPts`) directly from the Python GUI.
  - **Convex Hull Fill**: Integrated a convex hull post-processing step into the grid-based voting mechanism to eliminate holes and noise gaps within identified clusters.
  - **Performance Monitoring**: Implemented `avg_ms_per_tick` tracking. The average time per tick is now displayed in the window title and is available as a plottable variable in the time-series UI.

6.2.4 4. Unified Density Smoothing & Global Metrics (April 29th)

- **Objective**: Standardize density smoothing across all clustering modes and enable real-time plotting of global states.
- **Work Done**:
  - **Unified Smoothing**: Replaced redundant local smoothing in Auto K-Means with a global `human_density_smoothing_radius` parameter.
  - **Iterative Smoothing**: Added `human_density_smoothing_iterations` to allow for aggressive, multi-pass global smoothing.
  - **Global Metrics**: Exported `t_dynamic_state` and `t_tick_accumulators` to Python. Added variables like `k_fertility (sim)`, `phi_death_acc (sim)`, and various `death_* (sim)` debug counters to the UI plotting engine.
  - **Dual-Axis Fix**: Resolved a PyQtGraph closure bug where secondary axes caused the plot canvas to collapse.

6.2.5 5. Birth-Death Refactoring & Logic Consolidation (May 4th)

- **Objective**: Standardize parameter naming and consolidate biological logic for easier scientific review.
- **Work Done**:
  - **Parameter Schema Migration**: Replaced legacy `b1-b4` identifiers with standardized ecological parameters: `r` (growth rate), `NC` (carrying capacity), `Kmin`, and `Kmax`.
  - **Module Consolidation**: Migrated birth/death logic from the obsolete `mod_birth_death_new.f95` into the reviewed and consolidated `mod_reviewed_modules.f95`.
  - **Build System Sync**: Updated the dependency graph and Python interface to support the new parameter schema and modular structure.

6.2.6 6. Architectural Refinement & Critical Fixes (April - May)

- **K-Means Farthest-First Traversal**: Replaced the peak-value search in K-Means with a **Farthest First Traversal (K-means++)** subroutine to ensure geographically remote populations are isolated uniformly.
- **Cluster ID 1-Indexing**: Fixed a visualization bug where Cluster ID 0 was being masked as noise; the system now uses 1-based indexing for all valid clusters.
- **Rendering & Matrix Edge Geometry**: Resolved line rendering issues by translating cluster maps directly through NumPy boolean slicing filters and "burning" 1-pixel white boundaries into the RGBA output matrices.
- **3D Visualization**: Replaced unstable OpenGL 3D views with Matplotlib 3D surfaces in the `compare_smoothing.py` tool for robust cluster verification in headless environments.
- **F2PY Namespace Hotfix**: Fixed a critical bug where localized imports of `mod_python_interface` shadowed the extension module, causing `AttributeError` crashes in the update loop.

6.3 TODO later

- **[LATEX/DOCS]**: Add an entry explaining the Farthest-First Traversal algorithm.
- **[PYTHON/DOCS]**: Document that visual rendering loops for 2D and 3D are fundamentally separate.
- **[PYTHON/DOCS]**: Mark that the backend expects exactly 4-channel RGBA arrays for map updates.

6.3.1 7. Cluster Population Control & Multi-Population Graphing (May 13th)

- **Objective**: Resolve population convergence discrepancies between global and cluster-based modules and improve the Python graphing capabilities.
- **Work Done**:
  - **Divergence Analysis**: Identified that the cluster-based module (`new_birth`) was severely under-suppressing births compared to the global (`reviewed_birth`) module. This was caused by the cluster module relying on the per-cluster accumulator `n_alive_acc(jp)`, which structurally misses agents not assigned to any cluster (e.g. `c_idx = 0`), resulting in massive population under-estimation.
  - **Debug Tooling**: Implemented a global-vs-cluster diagnostic summation script within `python_interface.f95` to track the exact magnitude of unassigned agents escaping the control logic.
  - **Dynamic UI Plotting via Population Filters**:
  - Enhanced `mod_python_interface.get_dynamic_state_stats` and `get_cluster_info` to optionally accept and return stats based on a specific `jp` population index.
  - Added a dedicated `pop:` spinbox to the Python GUI (in `application.py`) that feeds into the graph configuration.
  - Implemented **"Dominant Population" (`pop = -1`) logic**, allowing graphs to dynamically query the backend tick-by-tick and automatically select and plot statistics (`NC`, `n_agents`, `k_fertility`) specifically for whichever population currently has the most agents within the targeted cluster or globally.

6.3.2 8. Performance Metrics HUD & Population-Specific Mating (May 26th)

- **Objective**: Add real-time wall-clock profiling to identify simulation bottlenecks, and introduce optional reproductive isolation between populations.
- **Work Done**:
  - **Fortran Instrumentation**: Wrapped each phase of `step_simulation` in `python_interface.f95` with `system_clock` calls to measure wall-clock time for permanent modules, active modules, compaction, grid/density, and clustering.
  - **API**: Exposed `set_performance_timing_enabled` and `get_performance_stats` to Python via `mod_python_interface`. Implemented as optional with **zero overhead** when disabled.
  - **GUI HUD**: Added a "Show Performance Metrics" checkbox in the "View Editor" tab and a glassmorphic tech-cyan overlay in the simulation window displaying per-phase timing breakdowns.
  - **Population-Specific Mating (`allow_across_populations`)**: Added a new config flag `allow_across_populations` (default `.true.`). When set to `.false.`, the `get_male_from_cell` function is called with `only_population=jp`, enforcing strict reproductive isolation so that females can only mate with males of the same population.
- **Files Modified**: `python_interface.f95`, `mod_agent_world.f95`, `mod_reviewed_modules.f95`, `mod_birth_death_new.f95`, `mod_config.f95`, `mod_read_inputs.f95`, `basic_config.nml`, `config_sandesh.nml`, `simulation.py`, `application.py`.

6.3.3 9. Critical Bugfix: Hashmap Population Corruption (May 26th)

- **Objective**: Fix a bug causing phantom agents of incorrect populations (e.g. Pop 1 red dots) to appear in single-population simulations.
- **Root Cause**: The `remove()` subroutine in `mod_hashmap.f95` used **open-address linear probing**. When an entry was deleted, subsequent entries in the probe chain were rehashed to fill the hole. The rehash code correctly saved and restored the `key` and `value` (array index) fields, but **failed to save or restore the `population` field**. The moved entry inherited a stale `population` value from the vacated bucket.
- **Impact**: When `get_agent(id, world)` later looked up a rehashed agent via `get_index_and_pop`, it received an incorrect population index. This caused an **out-of-bounds array access** into `agents(index, stale_population)`, reading garbage memory that could appear as an agent with `population = 1`, rendered as a red dot in the visualization.
- **Fix Applied** (4 changes to `mod_hashmap.f95`):
  - Added `temp_population` variable to the rehash loop.
  - Clear `population = -1` when initially removing an entry's bucket.
  - Save `temp_population` alongside `temp_key`/`temp_value` when reading entries to move.
  - Write `temp_population` to the destination bucket and clear `population = -1` on the source bucket after moving.
- **File Modified**: `src/data_structures/mod_hashmap.f95`.

6.3.4 10. Efficient Cluster-Restricted Density Updates (May 26th)

- **Objective**: Speed up the main bottleneck of the simulation step (the $O(N)$ grid sweeps for raw density, combined agent flows, box smoothing, and carrying capacity copies) by restricting updates to cells that belong to active clusters.
- **Work Done**:
  - **New Subroutine** `update_density_and_hep_grid_efficient`: Implemented a cluster-restricted variant of the full density update in `mod_technical_modules.f95`. Instead of sweeping all $n_x \times n_y$ cells, it iterates only over cells belonging to active clusters (`cluster_store%clusters(k)%cell_gx/gy`), computing raw density, agent flows, and HEP copies exclusively for those cells.
  - **Hybrid Scheduling in** `step_simulation`: Modified the density update dispatch in `python_interface.f95` to choose between the full and efficient variants based on tick alignment. On clustering ticks (`mod(t, update_interval) == 0`), the legacy `update_density_and_hep_grid` runs to ensure the full grid is populated before re-clustering. On intermediate ticks, the efficient variant is used.
  - **Boundary Orphan Reset**: Designed and implemented a `was_clustered(:,:)` boundary logical grid array inside the `Grid` type. On re-clustering ticks (`mod(t, update_interval) == 0`), any cell that transitioned OUT of a cluster is immediately zeroed out.
  - **O(1) Neighborhood box smoothing**: Restricted the smoothed density box-filter calculations to cluster cells, looking up neighbor cells directly (which naturally evaluate to `0.0` outside clusters).
  - **O(1) HEP data copies**: Restricted carrying capacity copies strictly to cells within active clusters.
  - **Conditional Startup Fallback**: Embedded conditional fallback inside `step_simulation` in `python_interface.f95` to automatically pivot to the legacy `update_density_and_hep_grid` on tick 0 or when no clusters exist, ensuring robust carrying capacity (`NC`) initialization and preventing agent die-offs.
  - **Config Flag**: Gated behind `efficient_density_updates` (boolean) in the namelist. Currently set to `.false.` in both `basic_config.nml` and `config_sandesh.nml` pending validation that it does not affect simulation accuracy.
- **Status**: Implemented and compiles, but **currently disabled** (`efficient_density_updates = .false.`).
- **Files Modified**: `python_interface.f95`, `mod_technical_modules.f95`, `mod_grid_id.f95`, `mod_config.f95`, `mod_read_inputs.f95`, `basic_config.nml`, `config_sandesh.nml`.

6.3.5 11. Responsive Rolling Average for Performance HUD (May 26th)

- **Objective**: Upgrade the performance metrics HUD from a global cumulative average to a responsive rolling average that forgets the distant past and immediately reacts to mid-run toggle switches.
- **Work Done**:
  - **20-Tick Sliding Window Buffer**: Designed and implemented a module-level sliding circular buffer array `perf_history(6, 20)` in `python_interface.f95` that records the exact timings of the last 20 ticks.

  - **Reset on Toggle**: Programmed the rolling timing history array to completely clear and restart fresh whenever performance timing is enabled or disabled in `set_performance_timing_en`
  - **Arithmetic Average stats**: Updated `get_performance_stats` to calculate the exact, unweighted arithmetic average of the recorded window (up to 20 samples), returning clean rolling averages without any clearing side-effects.
- **Files Modified**: `src/interfaces/python_interface.f95`.

6.3.6 12. HEP Data Chunked Loading (May 26th)

- **Objective**: Reduce memory footprint for large HEP NetCDF files (≈ 900 MB) by loading only a window of time slices instead of the entire dataset.
- **Work Done**:
  - **Dynamic Chunk Sizing**: `read_hep_data` in `mod_read_inputs.f95` now queries the HEP file size and calculates a `slices_per_chunk` count targeting ≈ 10 MB chunks. Only the first chunk is loaded at startup.
  - **On-Demand Re-Loading**: Added `load_hep_chunk_from_file` subroutine. When `t_hep` advances past the current chunk's range (`chunk_start_t..chunk_end_t`), a new chunk is loaded from the NetCDF file with the correct `start` offset.
  - **Local Index Translation**: `update_density_and_hep_grid` translates the global `t_hep` to a local chunk index via `local_idx = t_hep - chunk_start_t + 1` before reading `grid%hep(:,:,jp,local_idx)`.
  - **Grid Type Extension**: Added `slices_per_chunk`, `chunk_start_t`, `chunk_end_t` metadata fields to the `Grid` type in `mod_grid_id.f95`, and a `config` pointer so the chunk loader can access file paths.
- **Files Modified**: `mod_read_inputs.f95`, `mod_grid_id.f95`, `mod_agent_world.f95`, `mod_technical_mod`

6.3.7 13. Clustering Fix: "Clusters Way Too Small" (May 26th)

- **Objective**: Diagnose and fix the issue where watershed clusters were spatially too tight, capturing only $\approx 55\%$ of alive agents and missing $\approx 45\%$ in peripheral cells.
- **Root Cause**: Three interacting factors:
  1. `watershed_threshold` was changed from `0.00` $\rightarrow$ `0.05`: With the old value, every non-zero-density cell was included. The new value excluded peripheral cells where smoothed density fell below `0.05`.
  2. `human_density_smoothing_radius` was changed from `3` $\rightarrow$ `5`: The wider kernel spreads density further, reducing peak values and causing more edge cells to drop below threshold.
  3. Redundant internal smoothing in `watershed_cluster()`: The watershed algorithm was applying 4 additional iterations of box-filter smoothing (radius=4) on top of the already-smoothed `human_density_smoothed` surface.
- **Fix Applied**:
  - Lowered `watershed_threshold` to `0.01` in both `basic_config.nml` and `config_sandesh.nml`.
  - Removed the redundant 4-iteration internal smoothing loop from `watershed_cluster()` in `mod_watershed.f95`.
- **Files Modified**: `mod_watershed.f95`, `basic_config.nml`, `config_sandesh.nml`.

6.3.8 14. Bugfix: Out-of-Bounds Array Access in Cluster Accumulators (May 26th)

- **Objective**: Fix memory corruption caused by accessing `cell_cluster_idx` with invalid grid coordinates.
- **Root Cause**: Agents have a default `gx = -1` before being placed on the grid. The accumulator code in `update_agent_age` (`mod_technical_modules.f95`) accessed `cell_cluster_idx(agent%gx, agent%gy)` without bounds-checking. With `gx = -1`, this is an **out-of-bounds Fortran array access** that reads arbitrary memory.
- **Same bug in debug code**: The detailed debug breakup prints in `python_interface.f95` had the identical issue.
- **Fix Applied**:
  - Added `gx >= 1 .and. gx <= nx .and. gy >= 1 .and. gy <= ny` bounds check in `update_agent_age` before accessing `cell_cluster_idx`.
  - Added `c_idx <= n_clusters` upper-bound check before using `c_idx` as an index into `clusters(:)`.
  - Added matching bounds checks in the debug breakup code in `python_interface.f95`.
- **Files Modified**: `mod_technical_modules.f95`, `python_interface.f95`.

6.4 Tripwires & Lessons Learned

- **1. Fortran Array Out-of-Bounds is Silent**: Fortran does **not** bounds-check array accesses by default. Accessing `array(-1, 5)` doesn't crash — it silently reads garbage from adjacent memory. This garbage then propagates through the simulation as valid data.
- **2. Config Changes Can Have Non-Obvious Algorithmic Interactions**: The `watershed_threshold` increase from `0.00` to `0.05` seemed harmless — but it interacted catastrophically with the independently-added 4x internal smoothing in the watershed algorithm.
- **3. "Parity" Code Can Be Counter-Productive**: The internal smoothing loop was added to bring watershed clustering into "parity" with Auto K-Means. But the two algorithms have fundamentally different sensitivities.
- **4. Misleading Debug Output Can Send You in Circles**: The debug diagnostic showing "316 agents with invalid/unallocated indices" looked like a clustering algorithm failure. In reality, it was the debug code itself having an out-of-bounds bug.
- **5. `efficient_density_updates = .false.` Made Fix Attempts Into No-Ops**: The earlier fix attempt (hybrid density scheduling, `was_clustered` grid cleanup) was architecturally sound but was entirely guarded by `efficient_density_updates`, which was `.false.` in both configs.